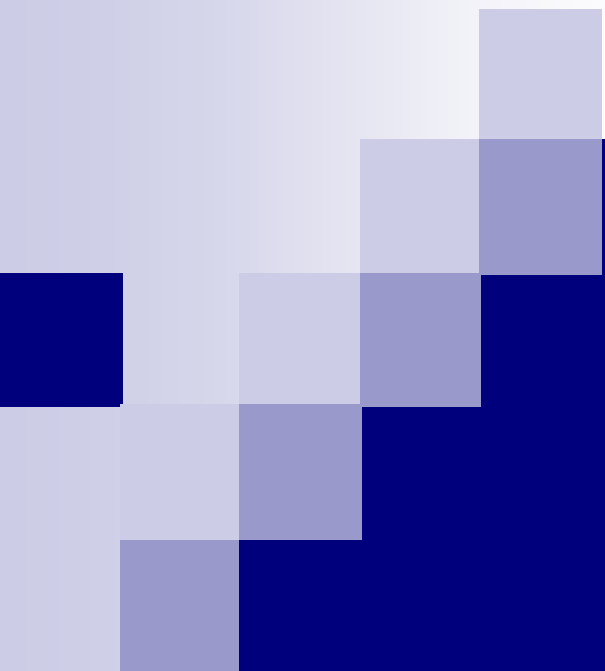




IT2164/IT2561

Operating Systems and Administration

Chapter 7

Deadlocks

Objectives

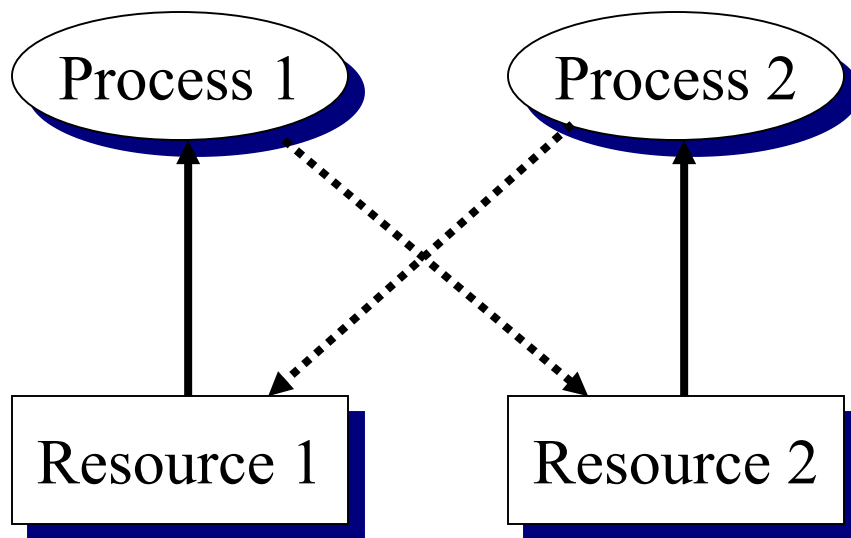
- After this lesson, you should be able to :
 - Understand deadlocks
 - Analyse deadlock situations using the 4 necessary conditions of deadlocks
 - Know the methods of deadlock prevention, avoidance, detection and recovery
 - Know how to apply Banker's algorithm to avoid deadlock and derive the safe sequence of a system

Deadlocks

- A deadlock is a situation where in a group of two or more processes/threads, each process holds at least one resource while making a request on another, and all requests cannot be satisfied because the requested resource is held by another process that is being blocked, waiting for another resource.

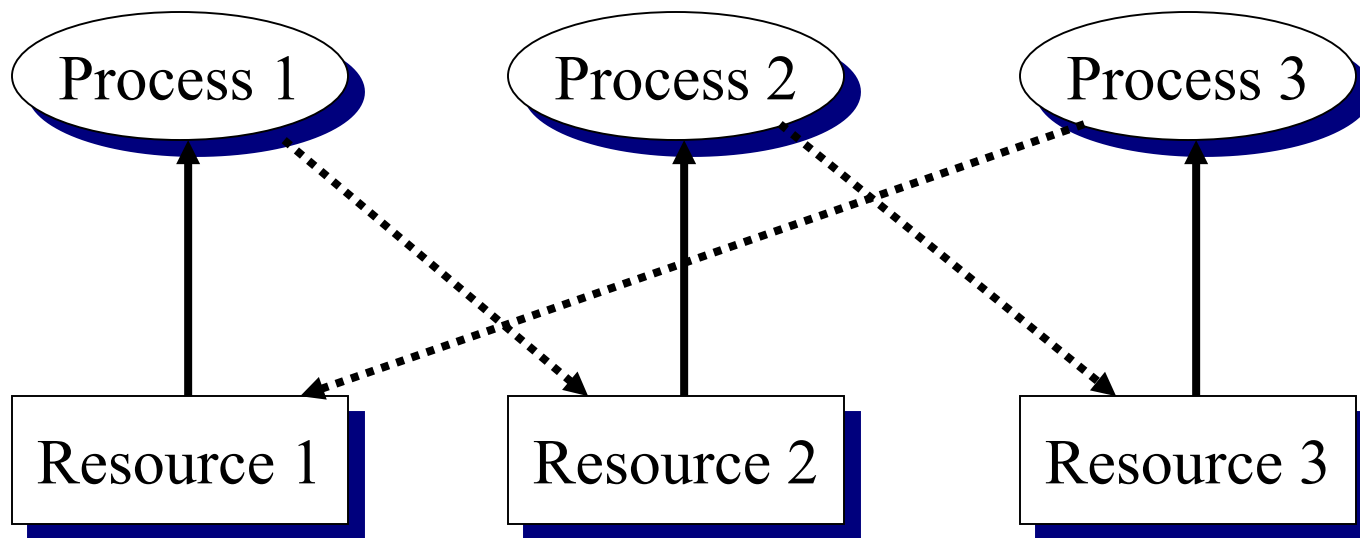
Dining Philosophers Problem

Example



- Process *holds* the resource
-→ Process *requests* the resource

Three Deadlocked Processes



- Process *holds* the resource
-→ Process *requests* the resource

When can deadlocks occur

- Deadlocks can occur in multiprogramming computer systems, which share resources.
- Deadlocks can also occur between threads in the same process.
- Resource managers (such as memory manager) can be involved in a deadlock with an application process.

Four conditions for deadlocks

- A computer system needs to have 4 conditions in order for deadlocks to be possible.
- Unfortunately, these 4 conditions occur in almost all resource allocation strategies used in modern OSs.
- Although these 4 conditions are necessary for deadlocks to occur, their presence is not sufficient to say that a deadlock exists.

Four Conditions

- Mutual exclusion (exclusive use)
 - Once a process has been allocated a particular resource, the threads in the process have exclusive use of the resource. No other processes can use a resource while it is allocated to a process.
- Hold and wait
 - A process may hold a resource at the same time it requests another one.

Four Conditions

■ Circular wait

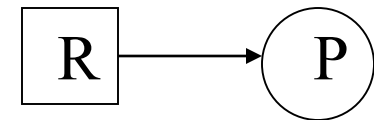
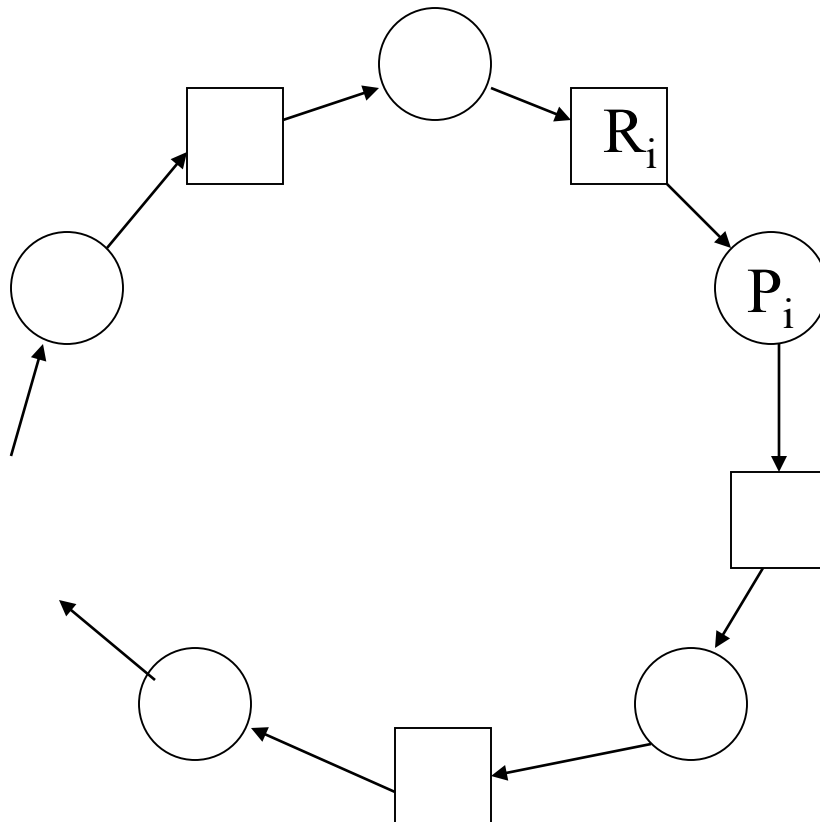
- A situation can rise in which process p_1 holds resource R_1 while one of its threads requests resource R_2 , and process p_2 holds R_2 while one of its threads requests resource R_1 . There may be more than two processes involved in the circular wait.

■ No preemption

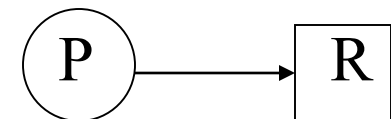
- Resources can be released only by the explicit action in a process, rather than by the action of an external authority. This assumption includes the case in which a process places a request for a resource and the resource is not available. Then the process cannot withdraw its request.

Circular Wait

- Have a situation in which there are K processes holding units of K resources



P holds R



P requests R

Addressing Deadlock

- There are 4 approaches to managing deadlock:
 - **Prevention:** Design the system so that deadlock is impossible
 - **Avoidance:** Construct a model of system states, then choose a strategy that will not allow the system to go to a deadlock state
 - **Detection & Recovery:** Check for deadlock (periodically or sporadically), then recover
 - **Manual intervention:** Have the operator reboot the machine if it seems too slow

Prevention

- Address the 4 conditions so that at least one of them does not hold.

①
②
③

 - Namely, mutual exclusion, hold and wait, circular waiting, no preemption. ④
- Most common strategy is to design the resource managers so that they are guaranteed to violate at least one of the conditions.
- Windows NT/2000 assures there can be no circular wait.

Prevention

■ Hold and Wait ²

□ Two ways:

- Require a process to request all of its resources when it is created.
 - Easy implementable in batch systems.
 - Not easy in interactive systems as processes do not 'announce' their resource requirements at runtime.
- Require the process to release all currently held resources prior to requesting any new ones.
 - More suitable for interactive systems.
 - Have processes request for resources based on a particular operation and then release it when proceeding onto another operation with a new set of resources.
 - Or, when a process requires a new resource, it releases all of its current resources and then acquires the old resources along with the new one.

Prevention

■ Circular Wait 3

□ Technique 1

- Use a process-resource model to detect cycles in the graph.
- Choose a resource request strategy by which no cycle will be introduced
- Cycle condition will reflect circular wait if each resource type has only one instance (non-sharable).

□ Technique 2

- Total order on all resources, then can only ask for R_j if $R_i < R_j$ for all R_i the process is currently holding
- Number each resource uniquely
- A process wishing to obtain resources must obtain them in ascending or decreasing order
- If new resource is not in the correct order, release all currently held resources and re-obtain them in the correct order.

Prevention

- 4
 - Allow preemption
 - If a process requests for a resource and the resource is not available, its currently held resources can be preempted by the OS and given to other resources.

Avoidance

- Manage deadlocks by allow resource allocation only if it is satisfied that deadlocks will not occur.
- The mechanism is **Banker's Algorithm**.

Detection and Recovery

- An aggressive strategy.
- Disregards deadlocks, but aims to detect when it *does* occur and takes additional steps to remove it. (recovery)
- Therefore, we need both a detection mechanism and a recovery mechanism.
- For detection, an algorithm similar to Banker's algorithm can be used.

Deadlock Detection

- When should the detection algorithm be invoked?
 1. Invoke each time a request for allocation cannot be granted immediately.
 - Advantage: able to identify the specific process that caused the deadlock
 - Disadvantage: overhead in computation time
 2. Invoke at regular time intervals, eg, once per hour
 3. Invoke when CPU utilization drops below 40%
- Note: Resources allocated to deadlocked processes will be idle until the deadlock is broken.

Deadlock Recovery

- Inform operator to deal with deadlock manually.
- Two options for breaking deadlock to recover system from deadlock automatically:
 1. Process termination
 2. Resource Preemption

1. Deadlock Recovery - Process Termination

- Abort all deadlocked processes
 - This method is expensive since processes may have computed for a long time, and all results must be discarded, and recomputed again.
- Abort one process at a time until the deadlock cycle is broken.
 - This method incurs overhead since after each process is aborted, deadlock detection algorithm must be invoked to check whether any process is still deadlocked.
- Note:
 - Aborting a process may not be simple, for example, process may be in the midst of updating a file
 - For partial termination method, a policy must be defined as to which process to terminate

1. Deadlock Recovery - Process Termination

- Factors to determine which process is chosen:
 - The priority of process
 - How long the process has computed and how much longer to go
 - How many and what type of resources the process has used
 - How many more resources the process needs in order to complete
 - How many processes will need to be terminated
 - Whether the process is interactive or batch.

2. Deadlock Recovery - Resource Preemption

- Preempt some resources from processes and give these resources to other processes until the deadlock cycle is broken:
 - Note: For both process termination or resource preemption methods, the system reclaims all resource allocated to the terminating processes.
- Three issues to be addressed:
 - **Selecting a victim**
 - Question: Which resources and processes to preempt?
 - Answer: Depends on cost, for example, amount of time process execute thus far, number of resources holding, etc.
 - **Rollback**
 - Question: If we preempt a resource from a process, what should be done to the process?
 - Answer: Either total rollback or rollback enough to break deadlock

2. Deadlock Recovery - Resource Preemption

□ Starvation

- Question: How to ensure starvation will not occur? That is, how can we guarantee that resources will not always be preempted from the same process?
- Answer: A common solution is to include the number of rollbacks in the cost factor.

Combined Approach to Deadlock Handling

- None of the basic approaches alone is appropriate to handle deadlock.
- For different classes of system resources, use different approaches:
 - Internal resources:
 - used by the system, for example, a PCB
 - prevention through resource ordering
 - Central memory:
 - memory used by a user job
 - prevention through preemption
 - Job resources:
 - devices that can be assigned, for example, tape drive, files.
 - Avoidance
 - Swappable space
 - Space on backing store for each user job
 - Preallocation

Conclusion

- Multiprogramming with shared resources introduce the problem of deadlocks.
- Need to ensure that deadlocks can be managed, without sacrificing too much performance.
- Different system requires different techniques due to different requirements.